תיעוד שיחת למידה עם ChatGPT

נושא: כתיבת בדיקות יחידה ב-Java באמצעות JUnit ובניית AppTest עבור App.java

מטרת המסמך:
תיעוד מסודר של שלבי הלמידה שהתנהלו מול ChatGPT בנושא בדיקות יחידה ב-Java, כולל השאלות שנשלחו, עיקרי התשובות, הדוגמאות שנלמדו, והאופן שבו התקדמה ההבנה עד לבניית בדיקות עבור הפונקציות בקובץ `App.java`.

הקשר לתרגיל:
במסגרת תרגיל הבית נדרש ללמוד בעזרת כלי LLM על כתיבת בדיקות יחידה בשפת Java, לתעד את שיחות הלמידה, ולאחר מכן להיעזר בכלי LLM כדי לכתוב בדיקות יחידה עבור הפונקציות בקובץ `App.java`. במהלך השיחה ניתן דגש על שימוש בפונקציות Assert שונות, על בדיקת מקרי קצה ועל בדיקת נתיבי הפונקציות.

תוכן העניינים

1. שלב 1 - בקשת הסבר בסיסי על בדיקות יחידה
2. שלב 2 - הבנת ההבדל בין בדיקה ידנית לבדיקת יחידה
3. שלב 3 - מבנה בדיקת יחידה בסיסית ב-Java
4. שלב 4 - למידה על Assert ועל סוגי בדיקות
5. שלב 5 - סיכום ביניים של החומר שנלמד
6. שלב 6 - מעבר מהסבר תאורטי לבניית AppTest
7. שלב 7 - התאמת הבדיקות לפונקציות בקובץ App.java
8. שלב 8 - בדיקת מספקות הבדיקות וכיסוי מקרי קצה
9. סיכום הלמידה

שלב 1 - בקשת הסבר בסיסי על בדיקות יחידה

הפנייה שנשלחה:
נשלחה בקשה לקבל הסבר מפורט ופשוט על כתיבת בדיקות יחידה בשפת Java.

עיקרי המענה:
הוסבר שבדיקת יחידה, או Unit Test, היא בדיקה אוטומטית שבודקת חלק קטן בקוד, בדרך כלל פונקציה אחת. המטרה היא לוודא שהפונקציה מחזירה את התוצאה הנכונה עבור קלט מסוים.

הוצגה ההבחנה בין הקוד הרגיל לבין קוד הבדיקה. למשל, אם קיימת פונקציה שמחברת שני מספרים:

```
public static int add(int a, int b) {
    return a + b;
}
```

ניתן לכתוב לה בדיקת יחידה שבודקת האם התוצאה שמתקבלת אכן נכונה:

```
@Test
public void testAdd() {
    int result = App.add(2, 3);

    assertEquals(5, result);
}
```

מה נלמד בשלב זה:

- בדיקת יחידה היא בדיקה אוטומטית.
- הבדיקה מתמקדת בדרך כלל בפונקציה אחת.
- בדיקה טובה משווה בין תוצאה צפויה לבין תוצאה שהתקבלה בפועל.
- ב-Java משתמשים לרוב בספריית JUnit לצורך כתיבת בדיקות יחידה.

שלב 2 - הבנת ההבדל בין בדיקה ידנית לבדיקת יחידה

הפנייה שנשלחה:
התבקשה הבהרה: מה ההבדל בין בדיקת יחידה לבין בדיקה רגילה שמתבצעת ידנית.

עיקרי המענה:
הוסבר שבבדיקה ידנית מריצים את התוכנית, מסתכלים על הפלט ובודקים בעיניים אם הוא נכון. לעומת זאת, בבדיקת יחידה כותבים קוד שבודק את הקוד באופן אוטומטי.

דוגמה לבדיקה ידנית:

```
System.out.println(App.add(2, 3));
```

במקרה כזה צריך להסתכל על המסך ולוודא שהודפס הערך 5. לעומת זאת, בדיקת יחידה מבצעת את הבדיקה בעצמה:

```
@Test
public void testAdd() {
    int result = App.add(2, 3);

    assertEquals(5, result);
}
```

בדיקה ידנית	בדיקת יחידה
בודקים בעיניים את התוצאה.	הקוד בודק את התוצאה אוטומטית.
איטית יותר כאשר יש הרבה פונקציות.	מאפשרת להריץ הרבה בדיקות בלחיצה אחת.
קל לשכוח לבדוק מקרים מסוימים.	הבדיקות נשמרות וניתן להריץ אותן שוב ושוב.
מתאימה לבדיקה כללית ומהירה.	מתאימה לבדיקת פונקציה מסוימת ומדויקת.

מה נלמד בשלב זה:

- בדיקה ידנית תלויה בבדיקה של המשתמש.
- בדיקת יחידה מחליטה באופן אוטומטי אם התוצאה נכונה.
- JUnit מציג בדיקה שעברה בצבע ירוק ובדיקה שנכשלה בצבע אדום.
- בדיקות יחידה עוזרות לזהות שגיאות אחרי שינויים בקוד.

שלב 3 - מבנה בדיקת יחידה בסיסית ב-Java

הפנייה שנשלחה:
התבקשה דוגמה לבדיקה בסיסית ב-Java, כולל הסבר על `@Test`, שם הפונקציה וגוף הבדיקה.

עיקרי המענה:
הוסבר שבדיקת יחידה בסיסית כוללת סימון `@Test`, פונקציה מסוג `void`, שם שמסביר מה נבדק, וגוף בדיקה שמפעיל פונקציה ומשתמש ב-Assert.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class AppTest {

    @Test
    public void testAddPositiveNumbers() {
        int result = App.add(3, 5);

        assertEquals(8, result);
    }
}
```

הסבר על `@Test`

הסימון `@Test` הוא Annotation של JUnit. הוא מסמן שהפונקציה שמתחתיו היא פונקציית בדיקה. ללא הסימון הזה, JUnit לא יריץ את הפונקציה כבדיקה.

הסבר על שם פונקציית הבדיקה

שם הפונקציה צריך לתאר מה בדיוק נבדק. למשל:

```
testAddPositiveNumbers
testFactorialOfZero
testIsPrimeWithPrimeNumber
```

הוסבר ששם כמו `test1` פחות טוב, כי הוא לא מסביר מה מטרת הבדיקה.

הסבר על גוף הבדיקה

הגוף של בדיקת יחידה בנוי בדרך כלל משלושה שלבים:

1. **Arrange** - הכנת הנתונים לבדיקה.
2. **Act** - הפעלת הפונקציה שרוצים לבדוק.
3. **Assert** - בדיקה שהתוצאה שהתקבלה היא התוצאה הצפויה.

```
@Test
public void testAddPositiveNumbers() {
    // Arrange
    int a = 3;
    int b = 5;

    // Act
    int result = App.add(a, b);

    // Assert
    assertEquals(8, result);
}
```

> **מה נלמד בשלב זה:**
>
> - `@Test` מסמן ל-JUnit שזו בדיקה.
> - פונקציית בדיקה היא בדרך כלל `public void`.
> - שם הבדיקה צריך להיות ברור ומתאר.
> - המבנה המקובל הוא Arrange, Act, Assert.

שלב 4 - למידה על Assert ועל סוגי בדיקות

> **המשך הלמידה:**
> במהלך השיחה הוסברו פונקציות Assert שונות והמשמעות של כל אחת מהן.

עיקרי המענה:
הוסבר כי Assert הוא משפט בדיקה. הוא בודק האם הערך שהתקבל בפועל תואם למה שהיה צפוי לקבל.

פונקציית Assert	מטרה	דוגמה
`assertEquals`	בודקת ששני ערכים שווים.	`assertEquals(5, result);`
`assertTrue`	בודקת שהתוצאה היא true.	`assertTrue(App.isPrime(7));`
`assertFalse`	בודקת שהתוצאה היא false.	`assertFalse(App.isPrime(9));`
`assertNotNull`	בודקת שהתוצאה אינה null.	`assertNotNull(result);`
`assertNull`	בודקת שהתוצאה היא null.	`assertNull(result);`
`assertThrows`	בודקת שנזרקת חריגה.	`assertThrows(...)`

דוגמה ל-assertThrows

הוסבר כי כאשר פונקציה אמורה לזרוק חריגה במקרה של קלט לא תקין, משתמשים ב-`assertThrows`.

```
@Test
public void testFactorialNegativeNumberThrowsException() {
    assertThrows(IllegalArgumentException.class, () -> {
        App.factorial(-3);
    });
}
```

מה נלמד בשלב זה:

- Assert הוא כלי שמחליט אם בדיקה עברה או נכשלה.
- קיימות כמה פונקציות Assert, וכל אחת מתאימה לסוג אחר של בדיקה.
- כאשר מצפים לחריגה, משתמשים ב-`assertThrows`.
- בבדיקת ערכי `double` כדאי להשתמש בסטייה קטנה, למשל `0.001`.

שלב 5 - סיכום ביניים של החומר שנלמד

הפנייה שנשלחה:
התבקש סיכום של כל החומר שנלמד עד אותו שלב בנקודות.

עיקרי המענה:
התקבל סיכום מסודר של מושגי היסוד בבדיקות יחידה.

- בדיקת יחידה היא בדיקה אוטומטית לפונקציה קטנה בקוד.
- ב-Java משתמשים בדרך כלל ב-JUnit.
- הסימון `@Test` מציין שזו בדיקת יחידה.
- פונקציית בדיקה היא בדרך כלל `public void`.
- שם הבדיקה צריך להסביר מה היא בודקת.
- בדיקה טובה בנויה לפי Arrange, Act, Assert.
- `assertEquals` בודק שוויון.
- `assertTrue` בודק שהתוצאה היא אמת.
- `assertFalse` בודק שהתוצאה היא שקר.
- `assertThrows` בודק שנזרקת חריגה.
- חשוב לבדוק גם מקרי קצה ולא רק מקרים רגילים.

דגש לימודי:
בשלב זה נוצר מעבר מהבנה כללית של בדיקות יחידה להבנה מעשית של איך כותבים אותן בפועל.

שלב 6 - מעבר מהסבר תאורטי לבניית AppTest

הפנייה שנשלחה:
נשלחה בקשה לעזור בבניית בדיקות יחידה לפונקציות שנמצאות בקובץ `java.App`, תוך שימוש בפונקציות Assert שונות בקובץ `AppTest`.

עיקרי המענה:
הוסבר כי יש ליצור קובץ בדיקות בשם `AppTest.java`, ובו לכתוב בדיקות עבור הפונקציות שנמצאות בקובץ `App.java`.

הוצגה תבנית כללית לקובץ הבדיקות:

```
package org.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class AppTest {

    @Test
    public void testSomething() {
        // Arrange

        // Act
```

```
        // Assert
    }
}
```

בנוסף הוסבר שכדי שהבדיקות יהיו טובות, עליהן לכלול:

- בדיקות למקרים רגילים.
- בדיקות למקרי קצה.
- בדיקות לקלטים לא תקינים.
- בדיקות לכל נתיבי התנאים בפונקציות.
- שימוש בכמה פונקציות Assert שונות.

שלב 7 - התאמת הבדיקות לפונקציות בקובץ App.java

הפנייה שנשלחה:
נשלח קוד של הקובץ `App.java`, שכלל פונקציות מתמטיות, פונקציות לעיבוד מחרוזות, רשימות ומפות.

עיקרי המענה:
לאחר קבלת הקוד, הוצעו בדיקות יחידה עבור הפונקציות השונות בקובץ `App.java`.

פונקציה: add

נלמד לבדוק חיבור של מספרים חיוביים, מספרים שליליים וחיבור עם אפס.

```
@Test
public void testAddPositiveNumbers() {
    int result = App.add(3, 5);
    assertEquals(8, result);
}

@Test
public void testAddNegativeNumbers() {
    int result = App.add(-3, -5);
    assertEquals(-8, result);
}

@Test
public void testAddWithZero() {
    int result = App.add(7, 0);
    assertEquals(7, result);
}
```

פונקציה: isPrime

נלמד לבדוק מספר ראשוני, מספר לא ראשוני, מספרים קטנים מ-2, ואת המספר 2 כמקרה קצה.

```
@Test
public void testIsPrimeWithPrimeNumber() {
    assertTrue(App.isPrime(7));
}

@Test
public void testIsPrimeWithNotPrimeNumber() {
    assertFalse(App.isPrime(9));
}

@Test
public void testIsPrimeWithNumberLessThanTwo() {
    assertFalse(App.isPrime(1));
    assertFalse(App.isPrime(0));
    assertFalse(App.isPrime(-5));
}

@Test
public void testIsPrimeWithTwo() {
    assertTrue(App.isPrime(2));
}
```

פונקציה: reverse

נלמד לבדוק היפוך מחרוזת רגילה, מחרוזת ריקה, ולוודא שהתוצאה אינה null.

```
@Test
public void testReverseRegularString() {
    String result = App.reverse("hello");
    assertEquals("olleh", result);
}

@Test
public void testReverseEmptyString() {
    String result = App.reverse("");
    assertEquals("", result);
}

@Test
public void testReverseNotNull() {
    String result = App.reverse("abc");
    assertNotNull(result);
}
```

פונקציה: factorial

נלמד לבדוק פקטוריאל של מספר רגיל, פקטוריאל של 0, פקטוריאל של 1, וקלט שלילי שאמור לזרוק חריגה.

```
@Test
public void testFactorialRegularNumber() {
    int result = App.factorial(5);
    assertEquals(120, result);
}

@Test
public void testFactorialZero() {
    int result = App.factorial(0);
    assertEquals(1, result);
}

@Test
public void testFactorialNegativeNumberThrowsException() {
    assertThrows(IllegalArgumentException.class, () -> {
        App.factorial(-3);
    });
}
```

פונקציה: isPalindrome

נלמד לבדוק פלינדרום פשוט, מחרוזת שאינה פלינדרום, ומחרוזת עם רווחים וסימני פיסוק.

```
@Test
public void testIsPalindromeSimpleWord() {
    assertTrue(App.isPalindrome("level"));
}

@Test
public void testIsPalindromeNotPalindrome() {
    assertFalse(App.isPalindrome("hello"));
}

@Test
public void testIsPalindromeWithSpacesAndSymbols() {
    assertTrue(App.isPalindrome("A man, a plan, a canal: Panama"));
}
```

פונקציה: fibonacciUpTo

נלמד לבדוק סדרת פיבונאצ'י עד גבול מסוים, קלט 0, וקלט שלילי שאמור לזרוק חריגה.

```
@Test
public void testFibonacciUpToTen() {
    List<Integer> result = App.fibonacciUpTo(10);
    assertEquals(Arrays.asList(0, 1, 1, 2, 3, 5, 8), result);
}

@Test
public void testFibonacciUpToZero() {
    List<Integer> result = App.fibonacciUpTo(0);
    assertEquals(Collections.singletonList(0), result);
```

```
}

@Test
public void testFibonacciNegativeInputThrowsException() {
    assertThrows(IllegalArgumentException.class, () -> {
        App.fibonacciUpTo(-1);
    });
}
```

פונקציה: charFrequency

נלמד לבדוק ספירת תווים במחרוזת רגילה, מחרוזת ריקה ומפה שאינה null.

```
@Test
public void testCharFrequencyRegularString() {
    Map<Character, Integer> result = App.charFrequency("banana");

    assertEquals(3, result.get('a'));
    assertEquals(2, result.get('n'));
    assertEquals(1, result.get('b'));
}

@Test
public void testCharFrequencyEmptyString() {
    Map<Character, Integer> result = App.charFrequency("");
    assertTrue(result.isEmpty());
}
```

פונקציה: isAnagram

נלמד לבדוק שתי מילים שהן אנגרמה, שתי מילים שאינן אנגרמה, ומקרה עם רווחים ואותיות גדולות.

```
@Test
public void testIsAnagramTrue() {
    assertTrue(App.isAnagram("listen", "silent"));
}

@Test
public void testIsAnagramFalse() {
    assertFalse(App.isAnagram("hello", "world"));
}

@Test
public void testIsAnagramWithSpacesAndCapitalLetters() {
    assertTrue(App.isAnagram("Dormitory", "Dirty room"));
}
```

פונקציה: average

נלמד לבדוק ממוצע של מערך רגיל, מערך עם איבר אחד, מערך עם מספרים שליליים, ומערך ריק שאמור לזרוק חריגה.

```
@Test
public void testAverageRegularArray() {
    double result = App.average(new int[]{2, 4, 6, 8});
    assertEquals(5.0, result, 0.001);
}

@Test
public void testAverageEmptyArrayThrowsException() {
    assertThrows(IllegalArgumentException.class, () -> {
        App.average(new int[]{});
    });
}
```

פונקציה: filterEvens

נלמד לבדוק סינון מספרים זוגיים מתוך רשימה, רשימה בלי זוגיים, רשימה ריקה, ורשימה עם מספרים שליליים ו-0.

```
@Test
public void testFilterEvensRegularList() {
    List<Integer> result = App.filterEvens(Arrays.asList(1, 2, 3, 4, 5, 6));
    assertEquals(Arrays.asList(2, 4, 6), result);
}

@Test
public void testFilterEvensNoEvenNumbers() {
    List<Integer> result = App.filterEvens(Arrays.asList(1, 3, 5, 7));
    assertTrue(result.isEmpty());
}

@Test
public void testFilterEvensWithNegativeNumbers() {
    List<Integer> result = App.filterEvens(Arrays.asList(-4, -3, -2, -1, 0));
    assertEquals(Arrays.asList(-4, -2, 0), result);
}
```

פונקציה: mostCommonWord

נלמד לבדוק טקסט שבו מילה אחת מופיעה יותר מהאחרות, ולוודא שהפונקציה מחזירה את המילה הנפוצה ביותר.

```
@Test
public void testMostCommonWordRegularText() {
    String result = App.mostCommonWord("Java is fun. Java is powerful.");
    assertEquals("java", result);
}
```

```
@Test
public void testMostCommonWordWithCapitalLetters() {
    String result = App.mostCommonWord("Dog dog DOG cat");
    assertEquals("dog", result);
}
```

שלב 8 - בדיקת מספקות הבדיקות וכיסוי מקרי קצה

המשך השיחה:
לאחר כתיבת הבדיקות, הוסבר איך לבדוק האם הבדיקות מספקות והאם כל הנתיבים החשובים בקוד נבדקו.

עיקרי המענה:
הוסבר כי בדיקות מספקות הן בדיקות שלא בודקות רק מקרה רגיל, אלא גם מקרי קצה, קלטים לא תקינים וכל נתיב אפשרי בתנאי הפונקציה.

קריטריונים לבדיקות מספקות

- **מקרה רגיל:** קלט תקין ופשוט שמייצג שימוש רגיל בפונקציה.
- **מקרה קצה:** ערך גבולי כמו 0, 1, מחרוזת ריקה, רשימה ריקה או מספר שלילי.
- **קלט לא תקין:** מצב שבו הפונקציה אמורה לזרוק חריגה.
- **כיסוי תנאים:** אם בפונקציה יש `if` ו-`else`, יש לבדוק את שני הצדדים.
- **שימוש בכמה Assert:** כדי להראות הבנה של סוגי בדיקה שונים.

דוגמה לכיסוי נתיבים בפונקציה factorial

בדיקה	סוג מקרה	מה נבדק
`factorial(5)`	מקרה רגיל	שהפונקציה מחשבת נכון פקטוריאל רגיל.
`factorial(0)`	מקרה קצה	ש-0! מחזיר 1.
`factorial(1)`	מקרה קצה	ש-1! מחזיר 1.
`factorial(-3)`	קלט לא תקין	שנזרקת חריגה מסוג IllegalArgumentException.

דוגמה לכיסוי נתיבים בפונקציה isPrime

בדיקה	מה נבדק
`isPrime(7)`	מספר ראשוני.
`isPrime(9)`	מספר לא ראשוני שמתחלק במספר אחר.

`isPrime(2)`	המספר הראשוני הקטן ביותר.
`isPrime(1), isPrime(0), isPrime(-5)`	מספרים קטנים מ-2 שאינם ראשוניים.

מה נלמד בשלב זה:

- לא מספיק לבדוק רק קלט רגיל.
- צריך לבדוק ערכים גבוליים.
- צריך לבדוק מצבים שבהם אמורה להיזרק חריגה.
- צריך לעבור על הקוד ולזהות את כל נתיבי התנאים.
- בדיקות טובות נותנות ביטחון ששינוי עתידי בקוד לא יגרום לתקלה שלא שמים לב אליה.

סיכום הלמידה

במהלך השיחה עם ChatGPT נבנה תהליך למידה הדרגתי בנושא בדיקות יחידה ב-Java. תחילה הוסבר הרעיון הכללי של Unit Tests, לאחר מכן הובהר ההבדל בין בדיקה ידנית לבין בדיקה אוטומטית, ובהמשך נלמד המבנה המעשי של בדיקת יחידה באמצעות JUnit.

לאחר ההסבר התאורטי, הלמידה עברה לשלב מעשי: נבנו בדיקות עבור הפונקציות שבקובץ `App.java`, תוך שימוש בפונקציות Assert שונות. במהלך בניית הבדיקות הודגש הצורך לבדוק מקרי קצה, קלטים לא תקינים ונתיבים שונים בקוד.

הנושאים המרכזיים שנלמדו

- מהי בדיקת יחידה ומה מטרתה.
- מה ההבדל בין בדיקה ידנית לבדיקת יחידה.
- איך משתמשים ב-JUnit.
- מה המשמעות של `@Test`.
- איך נותנים שמות נכונים לפונקציות בדיקה.
- איך כותבים בדיקה לפי Arrange, Act, Assert.
- איך משתמשים ב-`assertEquals`, `assertTrue`, `assertFalse`, `assertNotNull` ו-`assertThrows`.
- איך מזהים מקרי קצה.
- איך בודקים חריגות.
- איך יודעים אם הבדיקות מספקות.
- איך בונים קובץ `AppTest.java` עבור קובץ `App.java`.

מסקנה:
השיחה סייעה להפוך את הדרישה הכללית של "כתיבת בדיקות יחידה" לשלבים מעשיים וברורים: הבנת

המושג, הכרת מבנה הבדיקה, שימוש ב-Assert, זיהוי מקרי קצה, ולבסוף כתיבת בדיקות בפועל עבור הפונקציות בקובץ `App.java`.